

LLM Unlearning In LLMs, machine unlearning is increasingly motivated by removing harmful capabilities of models (Liu et al., 2024a; Li et al., 2024a). Prior works have introduced a number of LLM unlearning techniques (Eldan & Russinovich, 2023; Li et al., 2024a; Lu et al., 2022; Yao et al., 2023; Chen & Yang, 2023; Ishibashi & Shimodaira, 2023; Yu et al., 2023; Wang et al., 2023; Wu et al., 2023; Zhang et al., 2023a; Yuan et al., 2023; Maini et al., 2024; Lu et al., 2024; Goel et al., 2022; Lo et al., 2024; Huang et al., 2024a; Liu et al., 2024b), but existing methods suffer from adversarial vulnerabilities (Lynch et al., 2024; Łucki et al., 2024). Here, we show that LAT can improve over unlearning techniques including state-of-the-art RMU (Li et al., 2024a).

3 Methods

Targeted latent adversarial training We can view an LLM with parameters θ , as a composition of two functions, $LLM_\theta(x_i) = (g_\theta \circ f_\theta)(x_i)$, where f_θ is a feature extractor which maps text to latent activations $\ell_i = f_\theta(x_i) \in \mathbb{R}^{s \times d}$ and g_θ maps those latent activations to output a probability distribution for sampling: i.e., $\hat{y}_i \sim P(y|g_\theta(\ell_i))$. We define an adversarial attack as a function α with parameters δ which modifies the LLM’s inputs or latent activations. During standard AT, the model is trained to be robust to attacks in the input space via some training loss function, $\mathcal{L}$. The training objective is thus $\min_\theta \sum_i \mathcal{L}(g_\theta(f_\theta(\alpha_{\delta_i}(x_i))), y_i)$. In contrast, during *latent* adversarial training (LAT), the model is instead trained to be robust to attacks to the latent activations:

$$\min_\theta \sum_i \mathcal{L}(g_\theta(\alpha_{\delta_i}(f_\theta(x_i))), y_i) \quad (1)$$

During *untargeted* LAT (e.g., Casper et al. (2024b)), the attacker seeks to steer the model *away* from the desired behavior on a training example (x_i, y_i) . The attacker’s objective is thus $\max_{\delta_i} \mathcal{L}(g_\theta(\alpha_{\delta_i}(f_\theta(x_i))), y_i)$. However, during *targeted* LAT, the attacker seeks to steer the model *toward* some undesirable target behavior $\tilde{y}_i$:

$$\min_{\delta_i} \mathcal{L}(g_\theta(\alpha_{\delta_i}(f_{\theta_1}(x_i))), \tilde{y}_i) \quad (2)$$

Training methods Performing basic targeted LAT requires a dataset of desirable behaviors $\mathcal{D}_{\text{desirable}}$ and a dataset of undesirable behaviors $\mathcal{D}_{\text{undesirable}}$. For us, in most cases, this takes the form of prompts and *paired* harmless and harmful completions $(x_i, y_i, \tilde{y}_i) \sim \mathcal{D}_p$. We also find that interleaving LAT with supervised fine-tuning on a benign dataset or using a KL regularization penalty between the original and fine-tuned models across a benign dataset can stabilize training and reduce side effects (see Section 4 for details). We refer to this *benign* dataset as $\mathcal{D}_b$. We attack the residual stream of transformer LLMs with L_2 -norm-bounded perturbations, calculated using projected gradient descent (PGD) (Madry et al., 2017). Because the model and attacker are optimized using different completions to prompts, we only perturb the positions in the residual stream corresponding to the prompt – see Figure 1. We found that perturbing the residual stream at *multiple layers* rather than a single layer, each with its own ϵ constraint typically yielded better results. After experimenting with different choices of layers (1, 2, 3, 4, 10, 16, 22, and 28), we found that the simple heuristic of selecting four evenly spaced layers worked well across models and experiments. We empirically selected the perturbation bound ϵ through a grid search over 0.5, 1.0, 2.5, 6.0, 10.0, choosing the value that provided maximal robustness against jailbreak attacks on Llama-2. Notably, these hyperparameters demonstrated good generalization, maintaining their effectiveness when held fixed across our subsequent experiments on different models and tasks.

4 Experiments

Our approach: augmenting fine-tuning and adversarial training methods with LAT Here, we experiment with targeted LAT for improving robustness to jailbreaks, unlearning undesirable knowledge, and removing backdoors. Across experiments, we show how LAT can be used to augment a broad range

Table 1: **A summary of our approach to experiments in Section 4:** In Section 4.1 - Section 4.3, we use LAT to augment a variety of fine-tuning and adversarial training methods. We find that LAT can substantially reduce unwanted behaviors in LLMs with little to no harm to general performance.

Goal	Method Augmented with LAT
Jailbreak Robustness (Section 4.1)	Refusal Training (RT)
	Embedding-Space Adversarial Training (Xhonneux et al., 2024)
Backdoor Removal (Section 4.2)	Direct Preference Optimization (DPO) (Rafailov et al., 2024)
Unlearning (Section 4.3)	Who’s Harry Potter (WHP) (Eldan & Russinovich, 2023)
	Gradient Ascent (GA) (Jang et al., 2022)
	Representation Misdirection for Unlearning (RMU) (Li et al., 2024a)

of state-of-the-art fine-tuning and adversarial training algorithms. Table 1 summarizes the methods we augment with targeted LAT.²

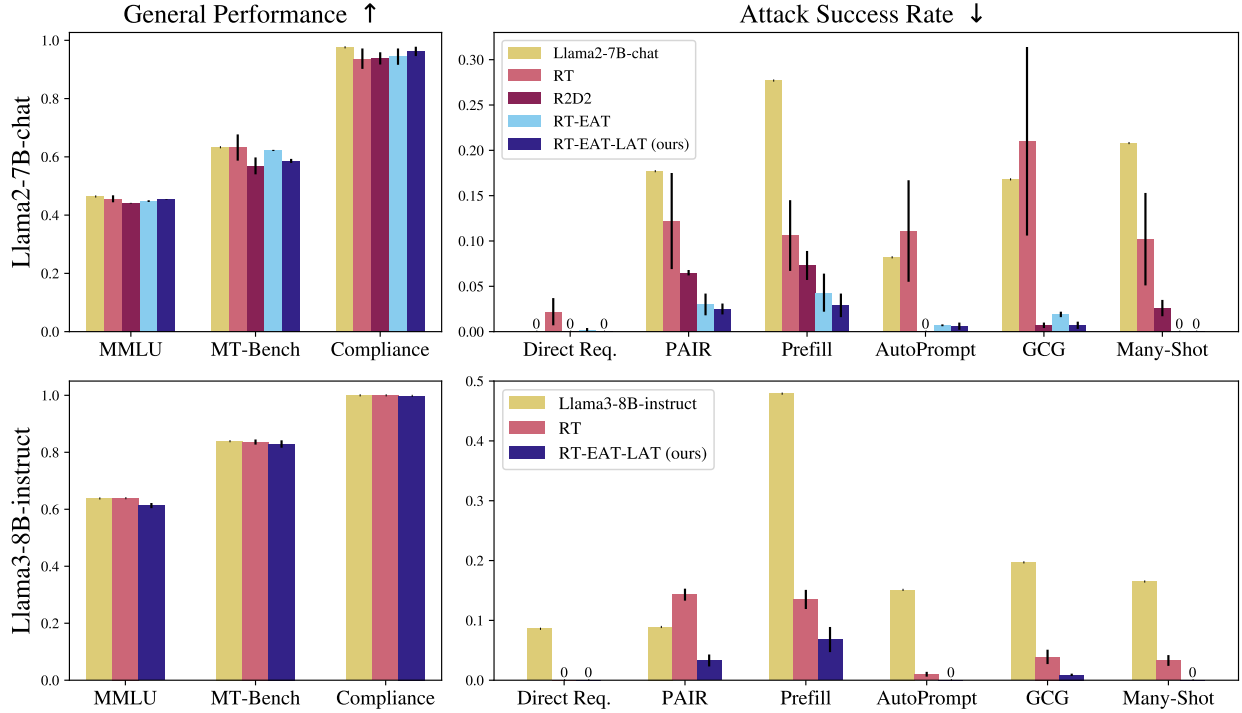
Our goal: improving the removal of undesirable behaviors with minimal tradeoffs to behavior in typical use cases. Because in different applications, practitioners may prefer different tradeoffs between performance in typical use cases and robust performance, we focus on the *Pareto frontier* between competing measures of typical performance and robustness to unwanted behaviors.

4.1 Improving Robustness to Jailbreaks

Data We create a dataset of triples containing: prompts, harmful completions, and harmless completions using a method based on Self-Instruct (Wang et al., 2022). We first generate a set of harmful user requests by few-shot prompting Mistral-7B (Jiang et al., 2023) with harmful requests seeded by AdvBench (Zou et al., 2023b). We then filter for prompts of an intermediate length and subsample for diversity by clustering BERT embeddings (Devlin et al., 2018) and sampling one prompt from each cluster. To generate harmful responses to the harmful user requests, we sampled from Zephyr-7B-Beta which was fine-tuned from Mistral-7B (Jiang et al., 2023) by Tunstall et al. (2023) to respond helpfully to user requests. We similarly generate refusals (harmless responses) using Llama2-7B-chat (Touvron et al., 2023) instruction-prompted to refuse harmful requests.

Model and methods Here, we fine-tune models using refusal training (RT). We implement refusal training based on Mazeika et al. (2024) using both a ‘toward’ and ‘away’ loss term calculated with respect to harmless/harmful example pairs. We then augment RT using three different techniques (see Appendix B for further details). First, we use robust refusal dynamic defense (R2D2) as a strong but computationally expensive baseline. Second, we augment RT using embedding-space (i.e. latent layer zero) adversarial training (RT-EAT) (Xhonneux et al., 2024). We refer to this as RT-EAT. Finally, we augment RT-EAT using LAT (RT-EAT-LAT). We perform LAT using latent-space adversaries at layers 8, 16, 24, and 30 which are jointly optimized to minimize the RT loss with the harmful/harmless labels flipped (see Appendix B.1). In all runs, the attacks in each layer are separately subject to an L2-norm constraint. In all experiments, we use the UltraChat dataset (Ding et al., 2023) as a benign fine-tuning dataset $\mathcal{D}_b$ to preserve the model’s performance. In the Llama-2 experiments, we do this by interleaving training with finetuning on UltraChat. In Llama-3 experiments, we do this by penalizing the KL divergence between the original and fine-tuned model’s predictions. Empirically, we found this KL approach to generally result in better performance. Finally, in Appendix C, we also compare our targeted LAT approach to untargeted LAT and find that untargeted LAT results in comparable performance to targeted LAT under some attacks and much worse performance under others.

²All experiments were run on a single A100 or H100 GPU except for ones involving R2D2 (Li et al., 2024a) in Section 4.1 which were run on eight. All training runs lasted less than 12 hours of wall-clock time.



Model	General Performance ↑			Attack Success Rate ↓						Relative Compute ↓
	MMLU	MT-Bench	Compliance	Direct Req.	PAIR	Prefill	AutoPrompt	GCG	Many-Shot	
Llama2-7B-chat	0.464	0.633	0.976	0.000	0.177	0.277	0.082	0.168	0.208	0x
RT	0.456 ±0.012	0.632 ±0.045	0.936±0.035	0.022±0.015	0.122±0.053	0.106±0.039	0.111±0.056	0.210±0.104	0.102±0.051	1x
R2D2	0.441±0.001	0.569±0.029	0.938±0.021	0.000 ±0.000	0.065±0.003	0.073±0.016	0.000 ±0.000	0.007 ±0.003	0.026±0.009	6558x
RT-EAT	0.448±0.003	0.622±0.002	0.944±0.028	0.002±0.002	0.030±0.012	0.043±0.021	0.007±0.001	0.019±0.003	0.000 ±0.000	9x
RT-EAT-LAT (ours)	0.454±0.001	0.586±0.007	0.962 ±0.016	0.000 ±0.000	0.025 ±0.006	0.029 ±0.013	0.006±0.004	0.007 ±0.004	0.000 ±0.000	9x
Llama3-8B-instruct	0.638	0.839	1.000	0.086	0.089	0.488	0.151	0.197	0.165	0x
RT	0.639 ±0.000	0.836 ±0.009	1.000 ±0.000	0.000 ±0.000	0.143±0.010	0.135±0.016	0.010±0.004	0.039±0.012	0.033±0.009	1x
RT-EAT-LAT (ours)	0.613±0.009	0.829±0.013	0.998±0.000	0.000 ±0.000	0.033 ±0.010	0.068 ±0.021	0.000 ±0.000	0.009 ±0.002	0.000 ±0.000	9x

Table 2: **LAT improves robustness to jailbreaking attacks with minimal side effects and small amounts of compute.** We compare LAT approaches to R2D2 (Mazeika et al., 2024) and embedding-space AT (EAT) (Xhonneux et al., 2024). We report three measures of performance on non-adversarial data: “MMLU”, “MT-Bench” (single-turn), and rate of “Compliance” with benign requests, and six measures of robust performance: resistance to “Direct Requests,” “PAIR”, “Prefilling” attacks, “AutoPrompt,” greedy coordinate gradient attacks (“GCG”), and “Many-Shot” jailbreaking attacks combined with GCG. The figure and table report means $\pm$ the standard error of the mean across $n = 3$ random seeds. Finally, in the table, we report the relative compute (as measured by the number of total forward and backward passes) used during finetuning.

Evaluation To evaluate the models’ performance in non-adversarial settings, we use the Massive Multitask Language Understanding (MMLU) benchmark, (Hendrycks et al., 2020), the MT-Bench benchmark (using a single-turn version) (Zheng et al., 2024), and the models’ rate of compliance with benign requests. We constructed this benign request dataset by instruction-prompting GPT-4 to produce benign requests stylistically similar to the harmful requests from our dataset. Similar to Liu et al. (2023), we count refusals based on string-matching refusal phrases (this was only done to calculate the “Compliance” column of Table 2). Next, to measure robustness, we use six attacks: direct requests with no adversarial optimization, prefilling attacks (Haizelabs), PAIR (Chao et al., 2023), AutoPrompt (AP) attacks (Shin et al., 2020), greedy coordinate gradient (GCG) attacks (Zou et al., 2023b), and many-shot jailbreaking attacks (Anil et al., 2024) combined with GCG. We evaluate the success of attacks using the StrongReject autograder (Souly et al.,